Each process has a */proc/[pid]/ns* sub-directory containing one entry for each namespace that supports being modified. Here's a simple example:

```
$ ps
  PID TTY          TIME CMD
 2551 pts/0    00:00:01 bash
15432 pts/0    00:00:00 ps

$ ls -al /proc/2551/ns
total 0
dr-x--x--x 2 saifi users 0 Apr  8 21:29 ./
dr-xr-xr-x 9 saifi users 0 Apr  8 12:24 ../
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 ipc ->
ipc:[4026531839]
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 mnt ->
mnt:[4026531840]
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 net ->
net:[4026531956]
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 pid ->
pid:[4026531836]
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 user ->
user:[4026531837]
lrwxrwxrwx 1 saifi users 0 Apr  8 21:29 uts ->
uts:[4026531838]
$
```

The corollary to this approach is the implementation of the *nsenter* program, which is part of the *util-linux* package, which lets a user run a program with the namespace of other processes.

Control groups in the Linux kernel

Since we also need access to kernel data structures, it is useful to work with a dynamic file system interface like */proc/mounts* to extract information about control groups (cgroups).

```
cgroup /sys/fs/cgroup/systemd cgroup rw,nosuid,nodev,noexec,r
elatime,xattr,release_agent=/usr/lib/systemd/systemd-cgroups-
agent,name=systemd 0 0
pstore /sys/fs/pstore pstore rw,nosuid,nodev,noexec,relatime
0 0
cgroup /sys/fs/cgroup/cpuset cgroup rw,nosuid,nodev,noexec,re
latime,cpuset 0 0
cgroup /sys/fs/cgroup/cpu,cpuacct cgroup rw,nosuid,nodev,noex
ec,relatime,cpu,cpuacct 0 0
cgroup /sys/fs/cgroup/memory cgroup rw,nosuid,nodev,noexec,re
latime,memory 0 0
cgroup /sys/fs/cgroup/devices cgroup rw,nosuid,nodev,noexec,r
elatime,devices 0 0
cgroup /sys/fs/cgroup/freezer cgroup rw,nosuid,nodev,noexec,r
elatime,freezer 0 0
cgroup /sys/fs/cgroup/net_cls,net_prio cgroup rw,nosuid,nodev
,noexec,relatime,net_cls,net_prio 0 0
```

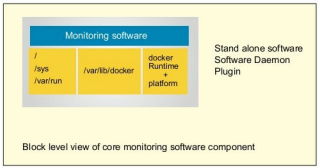

Figure 1: Docker monitoring architecture

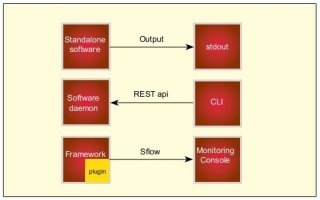

Figure 2: Deployment scenarios of a docker monitoring software

```
cgroup /sys/fs/cgroup/blkio cgroup rw,nosuid,nodev,noexec,rel
atime,blkio 0 0
cgroup /sys/fs/cgroup/perf_event cgroup rw,nosuid,nodev,noexe
c,relatime,perf_event 0 0
cgroup /sys/fs/cgroup/hugetlb cgroup rw,nosuid,nodev,noexec,r
elatime,hugetlb 0 0
```

This is what we need in order to collect system level metrics. Let's now focus on Docker.

Docker infrastructure

Docker implements a high-level API, which operates at the process level. To successfully monitor Docker container infrastructure, dynamic system-level access is required to the runtime, together with the systems view.

In short, we use the dynamic file system interface, and map it using volume functionality in the Docker runtime to collect the data that we seek. Thus, what we need is the following:

```
/
 /var/run
 /sys
/var/lib/docker
```

Figure 1 highlights the architectural blueprint for monitoring software, which is structured either as standalone software, as a software daemon or as a software plugin that is loaded and managed as part of a framework.